

1. Installation and Running via pip and uv

Module 5 supports reproducible installation through both pip and uv. The project declares its dependencies in `requirements.txt`, including Flask, SQLAlchemy, `psycopyg[binary]`, BeautifulSoup, urllib3, pytest, pytest-cov, Pylint, and pydeps. A `setup.py` file also defines the project as the installable package `jhu-software-concepts-module5`, identifies the Python modules under `src`, and declares the runtime dependencies required by the application.

For the pip workflow, I verified the installation from a separate temporary virtual environment rather than relying on packages already installed in the development environment. The environment was created and activated, `requirements.txt` was installed, and the project was installed in editable mode with `pip install -e ..`. I verified the resulting package with `pip show jhu-software-concepts-module5`, successfully imported all ten Module 5 modules, and ran the complete pytest suite successfully. The application can then be run from the Module 5 project using the installed dependencies and the required PostgreSQL environment configuration.

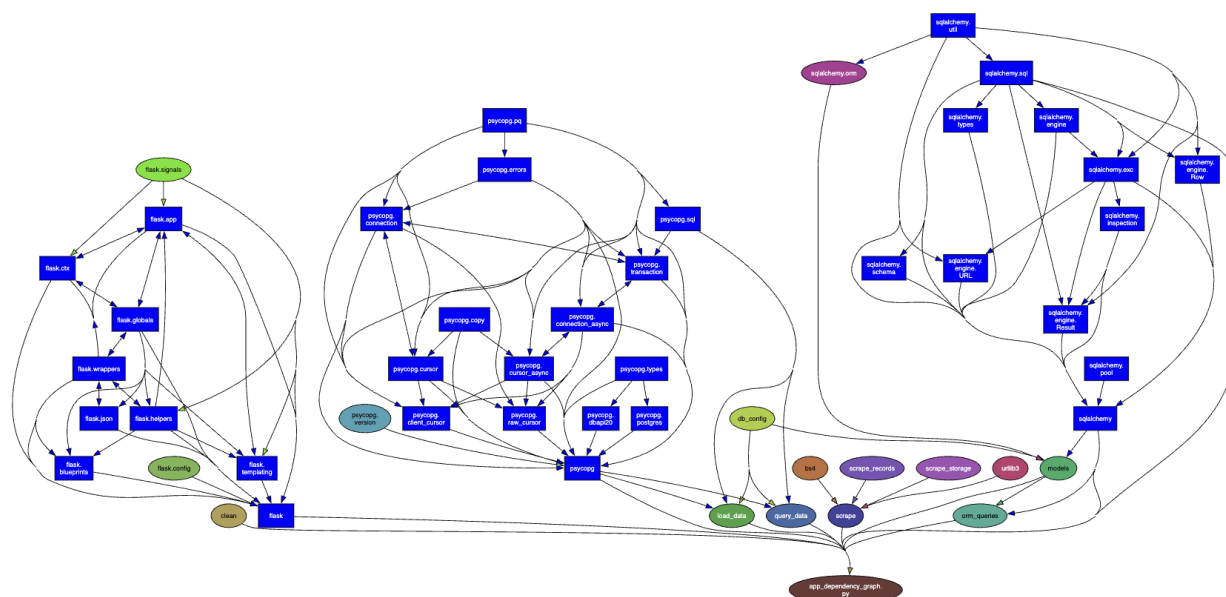
For the uv workflow, I performed a second fresh-install verification using a separate environment created with `uv venv step5_uv_venv`. After activating the environment, I synchronized it with the project's declared requirements using `uv pip sync requirements.txt`. The synchronization resolved and installed the nine dependencies declared in `requirements.txt`. I then installed the Module 5 project in editable mode using `uv pip install -e .`, which successfully built and installed `jhu-software-concepts-module5` version 1.0.0. I verified the installation by successfully importing all ten Module 5 modules and running the complete pytest suite. All 179 tests passed, and the ten source modules maintained 100% test coverage. This independently confirmed that the project can be installed and tested successfully using the uv workflow rather than only through the original pip environment.

Packaging the application is important because it makes the project installable, allowing imports to behave consistently across local execution, testing, and continuous integration rather than depending on the current working directory or manually configured Python paths. The `setup.py` file defines the project metadata, source modules, and runtime dependencies in a form that packaging tools can use. Editable installation through `pip install -e .` or `uv pip install -e .` also reduces “it works on my machine” path problems because the project is installed into the environment while remaining directly editable during development. Additionally, uv can use the project's packaging and dependency information when constructing and synchronizing environments, while `uv pip sync requirements.txt` ensures that the fresh environment matches the declared requirements for reproducibility.

2. Dependency Graph Summary

The Module 5 dependency graph was generated with pydeps and Graphviz to visualize both the project's internal architecture and its major external Python dependencies. The graph includes all ten Module 5 source modules—`app`, `clean`, `db_config`, `load_data`, `models`, `orm_queries`, `query_data`, `scrape`, `scrape_records`, and `scrape_storage`—so that standalone data-processing and database utilities are represented alongside the Flask application. The application layer connects `app` to `models`, `orm_queries`, and `scrape`, while the scraper is further separated into `scrape_records` and `scrape_storage`, reflecting the

modularization performed during the Pylint refactoring. The database portion of the graph shows `db_config` supplying configuration to database-dependent modules and displays the relationships with SQLAlchemy and psycopg, including representative submodules from each package. Flask is similarly expanded into components such as `flask.app`, `flask.ctx`, `flask.globals`, `flask.helpers`, `flask.templating`, and `flask.blueprints`, making the framework dependency structure visible rather than representing Flask as a single node. Low-level SQLAlchemy and psycopg implementation modules were selectively excluded from the visualization so that meaningful library structure could remain visible without allowing internal package dependencies to overwhelm the application's own architecture. Overall, the graph demonstrates that Module 5 separates web application, database configuration and querying, scraping, cleaning, loading, and analysis responsibilities while relying on Flask, SQLAlchemy, psycopg, BeautifulSoup, and urllib3 for major external functionality.



3. SQL Injection Defenses

Module 5 refactored the PostgreSQL query layer in `src/query_data.py` so that variable values are no longer inserted directly into raw SQL strings. SQL statements are constructed with psycopg's `sql.SQL()` interface, while dynamic SQL identifiers such as table and column names are represented using `sql.Identifier()`. Values are supplied separately through `%s` placeholders and parameter collections rather than through f-strings, string concatenation, or `.format()` applied to untrusted values.

For example, the query functions construct their SQL statement independently and then supply values such as "fall 2026" to the execution layer as parameters. The shared `fetch_one()` helper executes the composed statement using `cursor.execute(statement, params)` rather than combining the parameter values with the SQL text before execution. This distinction is important because psycopg sends the SQL structure and bound data separately, preventing supplied values from being interpreted as additional SQL syntax.

The same principle is used for dynamic identifiers. Identifiers cannot safely be supplied through ordinary value placeholders, so `sql.Identifier()` is used for identifiers while ordinary values continue to use `%s` parameters. This provides safe SQL composition while maintaining a clear boundary between the structure of a query and the data processed by that query.

4. Least-Privilege Database Configuration

Database connection information was moved out of the application source code and into environment variables. `src/db_config.py` reads and validates `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD`. A `.env.example` file documents the required configuration using placeholder values, while the real `.env` file is excluded from version control so that database credentials are not committed to the repository.

A dedicated PostgreSQL application role, `gradcafe_app`, was created instead of allowing the application to operate through the original PostgreSQL superuser. The role was configured with `NOSUPERUSER`, `NOCREATEDB`, and `NOCREATEROLE`. It was granted `CONNECT` on the `gradcafe` database and `USAGE` on the `public` schema. For the application's data operations, it has only `SELECT` and `INSERT` privileges on the `applicants` table and `USAGE` and `SELECT` on `applicants_p_id_seq`, which are required to support inserts using the table's sequence.

Representative privilege configuration:

```
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_app;
GRANT USAGE, SELECT ON SEQUENCE applicants_p_id_seq TO gradcafe_app;
```

The application role therefore has the permissions necessary for its normal database operations without receiving administrative privileges such as creating databases or roles or operating as a PostgreSQL superuser. Schema creation was also removed from the application's normal `load_data.main()` path because creating or altering database structure is an administrative/provisioning responsibility and would conflict with the least-privilege application role.

5. SQL Requirements: LIMIT, Separation, and Safe Composition

Module 5 enforces a maximum query result limit directly in `src/query_data.py`. The module defines `MAX_QUERY_LIMIT = 100`, and `clamp_limit()` constrains a requested limit to the permitted range of 1 through 100. All eleven `SELECT`-based question functions use `LIMIT %s`, and the shared `fetch_one()` helper applies the bound before execution and supplies the resulting limit as a parameter.

SQL statement construction and execution are also separated. Query functions construct composable SQL statements, while `fetch_one()` is responsible for executing those statements with their associated parameter values. The execution layer therefore receives the statement and parameters separately rather than receiving SQL text that has already had variable values interpolated into it.

Together, the implementation satisfies the three principal SQL-safety requirements: queries contain an enforced `LIMIT`, SQL construction is separated from execution, and `psycopg2`'s safe composition and parameterization mechanisms are used for identifiers and values. The resulting SQL layer prevents unrestricted result requests while also reducing SQL-injection risk.

6. GitHub Actions Continuous Integration

A GitHub Actions continuous integration workflow was implemented in `.github/workflows/ci.yml` and configured to run automatically on both pushes and pull requests. The workflow separates the required software-assurance checks into four independent jobs so that a failure in any individual quality or security requirement can be identified directly.

The Pylint job installs the Module 5 dependencies and runs `pylint --fail-under=10 src`. This enforces the assignment's required 10.00/10 Pylint score at the CI level rather than relying solely on local verification. During the initial CI run, GitHub Actions identified a naming issue involving the SQLAlchemy session factory that had not appeared in the local environment. The affected identifier was refactored from `SessionLocal` to `SESSION_LOCAL`, its references and corresponding tests were updated, and the project again achieved a 10.00/10 Pylint score.

The Dependency Graph job installs Graphviz and the project's Python dependencies before using `pydeps` to generate an SVG dependency graph. The workflow then executes `test -s` against the generated SVG so that the job fails if the dependency graph is not successfully created or is empty. This ensures that dependency visualization is continuously reproducible from the repository rather than existing only as a manually generated artifact.

The Snyk Dependency Scan job installs the Snyk CLI and executes `snyk test --org=rachmends` against Module 5. Authentication is supplied through the GitHub repository secret `SNYK_TOKEN`, so the Snyk credential is never stored directly in the workflow or committed to the repository. This reproduces the required dependency-security scan in the CI environment; the corresponding local Module 5 Snyk scan tested 30 dependencies and found no vulnerable dependency paths.

Finally, the Pytest job executes `pytest --cov=src --cov-report=term-missing --cov-fail-under=100`. Consequently, the CI job fails if any test fails or if source-code coverage falls below 100%. The initial GitHub Actions run also revealed that the clean CI environment did not contain the database environment variables available locally, so non-secret test configuration values were added to the Pytest job. After correcting the CI-specific Pylint and test-environment issues, the workflow was rerun successfully with all four jobs—Pylint 10.00, Dependency Graph, Snyk Dependency Scan, and Pytest—passing.

Figure: Successful GitHub Actions CI Run

rachmends / jhu_software_concepts

Q Type to search

<> CodeIssuesPull requestsAgentsActionsProjectsSecurity and qualityInsightsSettings

← Module 5 CI

✔ Fix Module 5 CI checks #2

Re-run all jobs

Summary

All jobs

✔ Pylint 10.00

✔ Dependency Graph

✔ Snky Dependency Scan

✔ Pytest

Run details

Usage

Workflow file

Triggered via push 1 minute ago

Status

Total duration

Artifacts

rachmends pushed - 53287f8 main

Success

42s

-

ci.yml

on: push

✔ Pylint 10.00

38s

✔ Dependency Graph

30s

✔ Snky Dependency Scan

30s

✔ Pytest

26s

-

+

Extra Credit (Synk Code Test)

Test Summary

```
Organization:
Test type:      Static code analysis
Project path:    /Users/rachrachmendmends/Desktop/JHU Python/jhu_software_concepts/Module 5

Total issues:    105
Ignored issues:  0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues:     105 [ 2 HIGH 39 MEDIUM 64 LOW ]
```

Summary:

Snyk Code successfully performed static application security testing on the local Module 5 project. The initial scan produced numerous findings because the project's committed virtual environment was included in the scan, causing Snyk Code to analyze third-party packages under `venv/lib/python3.11/site-packages`. Most reported issues therefore originated in external libraries rather than the Module 5 application source. Outside the virtual environment, Snyk identified two low-severity hardcoded-password warnings in test files and one medium-severity finding in `src/app.py` because the Flask development server was configured to run with debug mode enabled (`debug=True`). The test-file findings involve test configuration rather than production credentials. The debug-mode finding is directly associated with the Module 5 application and should be addressed by disabling debug mode in the application entry point.

Terminal Output:

Open Issues

✗ [LOW] Path Traversal

Finding ID: 39ca2d00-32e6-413b-b955-9470fb943333

Path: `venv/lib/python3.11/site-packages/sphinx/ext/autosummary/generate.py`, line 978

Info: Unsanitized input from a command line argument flows into `pathlib.Path`, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to remove arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 4e6bfd30-c4ae-4205-957a-6ca9729a7052

Path: `venv/lib/python3.11/site-packages/pip/_vendor/rich/syntax.py`, line 342

Info: Unsanitized input from a command line argument flows into `pathlib.Path`, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Code Injection

Finding ID: f618dafc-13c2-4bab-ae2a-44d015ef5957

Path: `venv/lib/python3.11/site-packages/_pytest/config/__init__.py`, line 913

Info: Unsanitized input from an environment variable flows into `importlib.import_module`, where it is executed as Python code. This may result in a Code Injection vulnerability.

✗ [LOW] Code Injection

Finding ID: c829727a-2ace-402b-9f4e-c4ffa7cdcba

Path: `venv/lib/python3.11/site-packages/_pytest/config/__init__.py`, line 917

Info: Unsanitized input from an environment variable flows into `__import__`, where it is executed as Python code. This may result in a Code Injection vulnerability.

✗ [LOW] Code Injection

Finding ID: 2983146d-7600-4ea7-ba06-af2db5b8f74b

Path: venv/lib/python3.11/site-packages/_pytest/config/__init__.py, line 885

Info: Unsanitized input from an environment variable flows into __import__, where it is executed as Python code. This may result in a Code Injection vulnerability.

✗ [LOW] Path Traversal

Finding ID: c720f7d8-e745-4a6d-acde-dc7d1c5baecf

Path: venv/lib/python3.11/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py, line 36

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 2d291c05-099f-4e93-894b-99583299c242

Path: venv/lib/python3.11/site-packages/dill/_objects.py, line 467

Info: hashlib.md5 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: fdc1520b-e9af-4f41-8068-267118511125

Path: venv/lib/python3.11/site-packages/pip/_vendor/distlib/database.py, line 1032

Info: hashlib.md5 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: b07002c5-a1d6-4ae1-8a57-41212129c999

Path: venv/lib/python3.11/site-packages/pip/_vendor/distlib/index.py, line 269

Info: hashlib.md5 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 1b8537c3-dcce-47fd-8df8-3bed93e66a6d

Path: venv/lib/python3.11/site-packages/pip/_vendor/requests/auth.py, line 148

Info: hashlib.md5 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 5e1b8d44-2584-4de1-914c-ac2ce116c8bd

Path: venv/lib/python3.11/site-packages/flask/sessions.py, line 281

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: a014a99e-bbff-4141-a7ab-4666bb6d056a

Path: venv/lib/python3.11/site-packages/itsdangerous/signer.py, line 45

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: c6e47946-90f7-477e-8734-df6dbfebbdc3

Path: venv/lib/python3.11/site-packages/jinja2/bccache.py, line 161

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 5d8638fa-c8fc-4735-8e0e-b8ebe2e599bc

Path: venv/lib/python3.11/site-packages/jinja2/bccache.py, line 165

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: b34c1bba-dc66-41e3-abd3-8466283a81bd

Path: venv/lib/python3.11/site-packages/jinja2/loaders.py, line 661

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 87673a26-b632-44be-8c9d-2aa93287b6b3

Path: venv/lib/python3.11/site-packages/pip/_vendor/requests/auth.py, line 156

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: b6d02d0a-d563-4359-9217-573bd1ae1ec3

Path: venv/lib/python3.11/site-packages/pip/_vendor/requests/auth.py, line 205

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 9acc9098-a62c-4b7d-811b-719170e2f110

Path: venv/lib/python3.11/site-packages/werkzeug/debug/__init__.py, line 45

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: 66db38b3-4cd2-420c-90ba-fc6d07cc68c2

Path: venv/lib/python3.11/site-packages/werkzeug/debug/__init__.py, line 205

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: a2d007f2-5f59-4ea8-bfb8-322e3dd905d8

Path: venv/lib/python3.11/site-packages/werkzeug/debug/__init__.py, line 211

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Use of Password Hash With Insufficient Computational Effort

Finding ID: e9288991-bbc2-45e7-a4e7-8b2f96ece5d7

Path: venv/lib/python3.11/site-packages/werkzeug/http.py, line 1077

Info: hashlib.sha1 is insecure. Consider changing it to a secure hashing algorithm.

✗ [LOW] Path Traversal

Finding ID: 791a27c7-c652-4245-8462-0a476a5b70e4

Path: venv/lib/python3.11/site-packages/_pytest/tmpdir.py, line 162

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path.

This may result in a Path Traversal vulnerability and allow an attacker to manipulate arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 7c33928a-665b-4c93-b534-0f7843ec55c3

Path: venv/lib/python3.11/site-packages/_pytest/_py/path.py, line 1426

Info: Unsanitized input from an environment variable flows into os.unlink, where it is used as a path.

This may result in a Path Traversal vulnerability and allow an attacker to remove arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 46a795ec-e40b-443a-83aa-bcb5e9bbf792

Path: venv/lib/python3.11/site-packages/sphinx/theming.py, line 552

Info: Unsanitized input from a command line argument flows into path concatenation, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 6ca8f097-5e43-4c3e-90e3-7e95d1edf0cd

Path: venv/lib/python3.11/site-packages/setuptools/command/easy_install.py, line 1955

Info: Unsanitized input from an environment variable flows into os.chmod, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to manipulate arbitrary files.

✗ [LOW] Path Traversal

Finding ID: fd836020-9115-4813-b17d-c259ac6f6fb9

Path: venv/lib/python3.11/site-packages/setuptools/command/easy_install.py, line 847

Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 0a02c0dc-2f5b-4b4c-bab1-806fa55beed9

Path: venv/lib/python3.11/site-packages/setuptools/command/easy_install.py, line 846

Info: Unsanitized input from an environment variable flows into os.unlink, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to remove arbitrary files.

✗ [LOW] Path Traversal

Finding ID: e4a30297-9bbd-4afc-804d-22696c545a2c

Path: venv/lib/python3.11/site-packages/pip/_vendor/pygments/cmdline.py, line 351

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: cbd8bc71-2929-4d94-817d-6278f9746712

Path: venv/lib/python3.11/site-packages/setuptools/command/easy_install.py, line 804

Info: Unsanitized input from an environment variable flows into os.walk, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 4d64263a-23bf-416c-a44e-62ec36630dbc

Path: venv/lib/python3.11/site-packages/setuptools/command/easy_install.py, line 617

Info: Unsanitized input from an environment variable flows into os.walk, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 065d1d0f-a1cd-44ac-9a2a-0dae8606f8d3

Path: venv/lib/python3.11/site-packages/sphinx/ext/autosummary/generate.py, line 702

Info: Unsanitized input from a command line argument flows into pathlib.Path, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: f6bddf9d-ad09-4ac3-ac7e-31825052ca0a

Path: venv/lib/python3.11/site-packages/platformdirs/windows.py, line 97

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: d94aa006-daa1-441b-b2ec-36008d7ad53b

Path: venv/lib/python3.11/site-packages/pip/_vendor/rich/json.py, line 135

Info: Unsanitized input from a command line argument flows into pathlib.Path, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: ddd3eee6-9d36-4a9f-9c87-5f9c45a97901

Path: venv/lib/python3.11/site-packages/sphinx/ext/autosummary/generate.py, line 715

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: e0a37180-55bd-4957-bb0e-67551948053a

Path: venv/lib/python3.11/site-packages/sphinx/cmd/quickstart.py, line 469

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: d56bece2-f00b-4c81-971b-a09076e355f5

Path: venv/lib/python3.11/site-packages/pip/_vendor/rich/emoji.py, line 96

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Use of Hardcoded Credentials

Finding ID: 6113c457-dc7e-4546-8181-dd9feb8f45f7

Path: venv/lib/python3.11/site-packages/pip/_internal/utis/misc.py, line 516

Info: Do not hardcode credentials in code. Found hardcoded credential used in here.

✗ [LOW] Use of Hardcoded Credentials

Finding ID: ef03690b-860d-46a5-9648-ddea13bcfc76

Path: venv/lib/python3.11/site-packages/_pytest/_py/path.py, line 1421

Info: Do not hardcode credentials in code. Found hardcoded credential used in here.

✗ [LOW] Use of Hardcoded Passwords

Finding ID: d15dda84-bc2a-49a5-8b14-10f3ad5c3dcd

Path: tests/test_load_data_coverage.py, line 302

Info: Do not hardcode passwords in code. Found hardcoded password used in a dictionary key.

✗ [LOW] Use of Hardcoded Passwords

Finding ID: dcf5f6ca-e4f0-44a2-b466-594bdccdd659

Path: tests/test_query_data_coverage.py, line 94

Info: Do not hardcode passwords in code. Found hardcoded password used in a dictionary key.

✗ [LOW] Path Traversal

Finding ID: 3da64185-8772-4136-9159-f2933573f13b

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/prepstyles.py, line 34

Info: Unsanitized input from a command line argument flows into zipfile.ZipFile, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 38d9418e-3158-43ef-b604-335ee06f3f6c

Path: venv/lib/python3.11/site-packages/pygments/cmdline.py, line 351

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: b3ed0f48-e8d9-4058-8a9f-a271c66bcf76

Path: venv/lib/python3.11/site-packages/flask/cli.py, line 1030

Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: d26cae1a-b8b7-44b7-9770-2118d61268bb

Path: venv/lib/python3.11/site-packages/flask/config.py, line 208

Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 9f35945e-6ef5-418e-b584-85e4cf6ccd87

Path: venv/lib/python3.11/site-packages/pylint/pyreverse/utils.py, line 41

Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 5b5a0060-2d23-4a3e-a7ac-9c1402ab1a77

Path: venv/lib/python3.11/site-packages/isort/main.py, line 96

Info: Unsanitized input from a command line argument flows into shutil.copypmode, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to manipulate arbitrary files.

✗ [LOW] Path Traversal

Finding ID: b82fb4fb-f6b5-4b28-98b5-7f6f04cbc2fe

Path: venv/lib/python3.11/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py, line 32

Info: Unsanitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: ea87d118-2a99-4395-8529-e50e91cc82a9

Path: venv/lib/python3.11/site-packages/isort/main.py, line 1078

Info: Unsanitized input from a command line argument flows into os.walk, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: e2d3873a-3bd1-4bba-bc00-1ae8b76e8fc1

Path: venv/lib/python3.11/site-packages/isort/main.py, line 1150

Info: Unsanitized input from a command line argument flows into os.walk, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 136ce7c7-2638-4fe5-a327-253900b0104d

Path: venv/lib/python3.11/site-packages/mccabe.py, line 300

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 6c98ac8e-02ac-4743-8986-3edbe5c7a5c7

Path: venv/lib/python3.11/site-packages/mccabe.py, line 309

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: b7b771ca-cdd5-457a-8881-c7b0ef8a3d67

Path: venv/lib/python3.11/site-packages/mccabe.py, line 311

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Insufficient postMessage Validation

Finding ID: 93e46024-4f00-461f-9a49-27a3f7e3ff7d

Path: venv/lib/python3.11/site-packages/urllib3/contrib/emscripten/emscripten_fetch_worker.js, line 12

Info: The origin of the received message is not checked. This means any site (even malicious) can send message to this window. If you don't expect this, consider checking the origin of sender.

✗ [LOW] Path Traversal

Finding ID: 67c98a84-34e7-45df-8a7b-87ec54a172be

Path: venv/lib/python3.11/site-packages/platformdirs/windows.py, line 106

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 19e53d00-4c64-4d1d-a3e4-7b5671b3c694

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/prepstyles.py, line 66

Info: Unsanitized input from a command line argument flows into shutil.move, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: b93e09a6-8705-4f96-8921-202aae19120f

Path: venv/lib/python3.11/site-packages/sphinx/cmd/quickstart.py, line 480

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 5ee77fd4-f0ae-469c-82d9-7cbc930ca97c

Path: venv/lib/python3.11/site-packages/sphinx/ext/autosummary/generate.py, line 645

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: f057a1ed-503e-4513-81f8-b8441a294707

Path: venv/lib/python3.11/site-packages/sphinx/ext/autosummary/generate.py, line 748

Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 16dcb7c3-174f-48ea-ba52-f9f20a1897fe

Path: venv/lib/python3.11/site-packages/pip/_vendor/platformdirs/unix.py, line 43

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: ffa6dd7c-5416-4e73-a442-ccc61b62b0f8

Path: venv/lib/python3.11/site-packages/pip/_vendor/platformdirs/unix.py, line 74

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: d7ec94c2-b04e-4297-bdba-85216109fc26

Path: venv/lib/python3.11/site-packages/pip/_vendor/platformdirs/unix.py, line 98

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path.

This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: aaa374ec-1d61-4e93-87c1-eca6bb229f77

Path: venv/lib/python3.11/site-packages/pip/_vendor/platformdirs/unix.py, line 114

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path.

This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 95f64fcf-f933-41ad-850b-79a0f61017ea

Path: venv/lib/python3.11/site-packages/pip/_vendor/platformdirs/unix.py, line 167

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path.

This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [LOW] Path Traversal

Finding ID: 5d7de302-72ca-4b50-a7e4-77bb7a25382d

Path: venv/lib/python3.11/site-packages/platformdirs/windows.py, line 50

Info: Unsanitized input from an environment variable flows into pathlib.Path, where it is used as a path.

This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

✗ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 395c4087-8d2f-44a4-ac18-93790eadf0bb

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/__init__.py, line 2855

Info: xml.etree.ElementTree.fromstring is vulnerable to XXE and DDOS in versions of python <= 3.10.

If using python runtime 3.10 or older, consider upgrading python versions, or see

<https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✗ [MEDIUM] SQL Injection

Finding ID: 4f64e710-3f00-4c7e-906a-f095f8ba131c

Path: venv/lib/python3.11/site-packages/sqlalchemy/dialects/mssql/base.py, line 3294

Info: Unsanitized input from a database flows into execute, where it is used in an SQL query. This may result in an SQL Injection vulnerability.

✗ [MEDIUM] Code Injection

Finding ID: ffffff6b-b7c7-4fdd-ae46-8bf0f90d9e6c

Path: venv/lib/python3.11/site-packages/flask/config.py, line 209

Info: Unsanitized input from an environment variable flows into exec, where it is executed as Python code. This may result in a Code Injection vulnerability.

✗ [MEDIUM] Jinja auto-escape is set to false.

Finding ID: 15e9795c-02ed-4ebe-8ef0-961ebfb7e4e8

Path: venv/lib/python3.11/site-packages/setuptools/_vendor/pyparsing/diagram/__init__.py, line 48

Info: jinja2.Template is called with no autoescape argument (autoescaping is disabled by default). This increases the risk of Cross-Site Scripting (XSS) attacks.

✗ [MEDIUM] Jinja auto-escape is set to false.

Finding ID: 6aa99f39-a32c-4275-b7ef-681700398c91

Path: venv/lib/python3.11/site-packages/pkg_resources/_vendor/pyparsing/diagram/__init__.py, line 48
Info: jinja2.Template is called with no autoescape argument (autoescaping is disabled by default). This increases the risk of Cross-Site Scripting (XSS) attacks.

✘ [MEDIUM] Jinja auto-escape is set to false.

Finding ID: dd401d9e-e190-41f0-b02e-6df890f621d3

Path: venv/lib/python3.11/site-packages/pip/_vendor/pyparsing/diagram/__init__.py, line 55

Info: jinja2.Template is called with no autoescape argument (autoescaping is disabled by default). This increases the risk of Cross-Site Scripting (XSS) attacks.

✘ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 09ed2533-411d-4627-bde1-b5cb9e4b9a24

Path: venv/lib/python3.11/site-packages/sphinx/ext/graphviz.py, line 242

Info: xml.etree.ElementTree.parse is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✘ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 03649b7d-4861-4aa7-b78d-2243434b054f

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/prepstyles.py, line 39

Info: xml.etree.ElementTree.iterparse is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✘ [MEDIUM] Code Injection

Finding ID: 98ea8cea-5f4b-48ef-b43d-cb33a89bc8ab

Path: venv/lib/python3.11/site-packages/flask/cli.py, line 1031

Info: Unsanitized input from an environment variable flows into eval, where it is executed as Python code. This may result in a Code Injection vulnerability.

✘ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: bc85e19c-ac74-4981-9662-26dceb61e41d

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/__init__.py, line 721

Info: xml.dom.minidom.parseString is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✘ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 2a6b7584-cdde-488a-afb2-3b0dbf43f17f

Path: venv/lib/python3.11/site-packages/docutils/writers/docutils_xml.py, line 77

Info: xml.sax.make_parser is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✘ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 36973191-9de7-4cdb-b729-6a38293725d2

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/__init__.py, line 2633

Info: xml.etree.ElementTree.fromstring is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✗ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 32a20ae0-b448-45c3-9938-1fad7993ef79

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/__init__.py, line 952

Info: xml.etree.ElementTree.fromstring is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✗ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: a54e081d-7314-416d-93cb-f0c5fcc72435

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/__init__.py, line 947

Info: xml.etree.ElementTree.fromstring is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✗ [MEDIUM] Code Injection

Finding ID: 13cde252-80f4-4ec2-b66b-cdbd12811b79

Path: venv/lib/python3.11/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py, line 77

Info: Unsanitized input from an environment variable flows into importlib.import_module, where it is executed as Python code. This may result in a Code Injection vulnerability.

✗ [MEDIUM] DOM-based Cross-site Scripting (XSS)

Finding ID: 9322ff24-80a2-47c2-95d4-1aedaac73e03

Path: venv/lib/python3.11/site-packages/werkzeug/debug/shared/debugger.js, line 273

Info: Unsanitized input from data from a remote resource flows into innerHTML, where it is used to dynamically construct the HTML page on client side. This may result in a DOM Based Cross-Site Scripting attack (DOMXSS).

✗ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: bb300407-99d9-41c0-b5cc-882050e3499f

Path: venv/lib/python3.11/site-packages/docutils/writers/odf_odt/__init__.py, line 943

Info: xml.etree.ElementTree.fromstring is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✗ [MEDIUM] Insecure Xml Parser (Python < 3.11)

Finding ID: 5af7bc9e-2684-4f39-8457-70c1df480a35

Path: venv/lib/python3.11/site-packages/docutils/writers/_html_base.py, line 498

Info: xml.etree.ElementTree.fromstring is vulnerable to XXE and DDOS in versions of python <= 3.10. If using python runtime 3.10 or older, consider upgrading python versions, or see <https://docs.python.org/3.10/library/xml.html#xml-vulnerabilities> for mitigations. Python versions 3.11 and onwards do not have this issue.

✗ [MEDIUM] Insecure Temporary File

Finding ID: 92c3e328-8e9f-43cd-8c79-951f1da55348

Path: venv/lib/python3.11/site-packages/setuptools/_distutils/util.py, line 422

Info: Use of tempfile.mktemp is deprecated and poses a security risk

✗ [MEDIUM] Debug Mode Enabled

Finding ID: 743ae8ff-066c-4147-8fc9-aabcc24b89bc

Path: src/app.py, line 432

Info: Running the application in debug mode (debug flag is set to True in run) is a security risk if the application is accessible by untrusted parties.

✗ [MEDIUM] Server Information Exposure

Finding ID: bbb6188a-3337-4c97-aaab-1031db197740

Path: venv/lib/python3.11/site-packages/flask/app.py, line 1224

Info: The stack trace from an exception flows into response_class and is leaked to the user. This may disclose sensitive server information to the attackers.

✗ [MEDIUM] Regular Expression Denial of Service (ReDoS)

Finding ID: eddbbfe2-7bea-474a-acef-827e3413a6d0

Path: venv/lib/python3.11/site-packages/isort/main.py, line 1144

Info: Unsanitized user input from a command line argument flows into re.compile, where it is used to build a regular expression. This may result in a Regular expression Denial of Service attack (reDOS).

✗ [MEDIUM] Arbitrary File Write via Archive Extraction (Tar Slip)

Finding ID: 366285e8-88a8-4446-b9a0-f5e538c36b1f

Path: venv/lib/python3.11/site-packages/pip/_vendor/distlib/util.py, line 1310

Info: Unsanitized input from an opened tar file flows into extractall, where it is used to extract files from a tar archive. This may result files outside destination directory to be overwritten, resulting in an arbitrary file write.

✗ [MEDIUM] Arbitrary File Write via Archive Extraction (Tar Slip)

Finding ID: 92673052-e083-4ad6-8710-2bcd9a7608f1

Path: venv/lib/python3.11/site-packages/pygments/lexers/_php_builtins.py, line 3302

Info: Unsanitized input from an opened tar file flows into extractall, where it is used to extract files from a tar archive. This may result files outside destination directory to be overwritten, resulting in an arbitrary file write.

✗ [MEDIUM] Arbitrary File Write via Archive Extraction (Tar Slip)

Finding ID: 2ad0cd1d-dad3-4407-8ef0-3414c4b1ce34

Path: venv/lib/python3.11/site-packages/pygments/lexers/_php_builtins.py, line 3304

Info: Unsanitized input from an opened tar file flows into extractall, where it is used to extract files from a tar archive. This may result files outside destination directory to be overwritten, resulting in an arbitrary file write.

✗ [MEDIUM] Command Injection

Finding ID: f928f12d-03f1-42a6-850f-55131a52e724

Path: venv/lib/python3.11/site-packages/click/_termui_impl.py, line 729

Info: Unsanitized input from an environment variable flows into subprocess.Popen, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: f5fa40cf-724d-4376-8c93-9d245e32b127

Path: venv/lib/python3.11/site-packages/sphinx/cmd/make_mode.py, line 177

Info: Unsanitized input from an environment variable flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Code Injection

Finding ID: e588c60b-dcf3-4a02-837c-772058bd5bdd

Path: venv/lib/python3.11/site-packages/setuptools/launch.py, line 32

Info: Unsanitized input from a command line argument flows into exec, where it is executed as Python code. This may result in a Code Injection vulnerability.

✗ [MEDIUM] Regular Expression Denial of Service (ReDoS)

Finding ID: 6463b9fd-0908-439c-a03b-bffe76fc7f39

Path: venv/lib/python3.11/site-packages/coverage/config.py, line 632

Info: Unsanitized user input from an environment variable flows into re.compile, where it is used to build a regular expression. This may result in a Regular expression Denial of Service attack (reDOS).

✗ [MEDIUM] Command Injection

Finding ID: ef3eeee7-f2df-42e8-a6b6-cba031084512

Path: venv/lib/python3.11/site-packages/werkzeug/_reloader.py, line 275

Info: Unsanitized input from a command line argument flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Regular Expression Denial of Service (ReDoS)

Finding ID: 9927ae66-8c59-495c-adfc-c35abfc216a7

Path: venv/lib/python3.11/site-packages/sphinx/ext/apidoc/_cli.py, line 263

Info: Unsanitized user input from a command line argument flows into re.compile, where it is used to build a regular expression. This may result in a Regular expression Denial of Service attack (reDOS).

✗ [MEDIUM] Command Injection

Finding ID: 117d1f10-1775-4edb-8913-879991aae3d0

Path: venv/lib/python3.11/site-packages/setuptools/_distutils/_msvccompiler.py, line 84

Info: Unsanitized input from an environment variable flows into subprocess.check_output, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: 0a8c1908-d3db-438e-91a2-9191ab03db13

Path: venv/lib/python3.11/site-packages/pip/_internal/commands/configuration.py, line 239

Info: Unsanitized input from an environment variable flows into subprocess.check_call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: 80370e00-75c8-4099-83f1-23b190c5cb27

Path: venv/lib/python3.11/site-packages/pip/_internal/cli/main_parser.py, line 90

Info: Unsanitized input from an environment variable flows into subprocess.run, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: b3ea0fd4-1ee2-46fa-951d-3a947720d09d

Path: venv/lib/python3.11/site-packages/click/_termui_impl.py, line 660

Info: Unsanitized input from an environment variable flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: 0b4e3331-d751-499a-a7a1-d158aa702f7d

Path: venv/lib/python3.11/site-packages/sphinx/cmd/make_mode.py, line 117

Info: Unsanitized input from an environment variable flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: c8230c14-7482-49d8-a5a4-3bd5a83f59cf

Path: venv/lib/python3.11/site-packages/sphinx/cmd/make_mode.py, line 132

Info: Unsanitized input from an environment variable flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: 1c754523-0980-48aa-97bb-a8f643ce0020

Path: venv/lib/python3.11/site-packages/sphinx/cmd/make_mode.py, line 145

Info: Unsanitized input from an environment variable flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [MEDIUM] Command Injection

Finding ID: 2a3e4883-cc5d-40a7-bcb3-e8eb2916b0e5

Path: venv/lib/python3.11/site-packages/sphinx/cmd/make_mode.py, line 162

Info: Unsanitized input from an environment variable flows into subprocess.call, where it is used as a shell command. This may result in a Command Injection vulnerability.

✗ [HIGH] Cross-site Scripting (XSS)

Finding ID: 4ddf4e12-a5c0-4caa-893c-6bab730741e7

Path: venv/lib/python3.11/site-packages/werkzeug/serving.py, line 313

Info: Unsanitized input from the HTTP request body flows into write, where it is used to render an HTML page returned to the user. This may result in a Cross-Site Scripting attack (XSS).

✗ [HIGH] Selection of Less-Secure Algorithm During Negotiation (SSL instead of TLS)

Finding ID: 1385092a-a38b-45c6-ba0a-4f018d72287c

Path: venv/lib/python3.11/site-packages/pip/_vendor/distlib/util.py, line 1496

Info: Consider using TLS instead of SSL (ssl.PROTOCOL_SSLv23 used in ssl.SSLContext).